CS 6515: Graduate Algorithms

Lecture 1: Course Overview, Asymptotic Notation, and Merge Sort

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

1 Announcements

1. PhD students on waitlist should email Sahil (ssingla@gatech.edu) their gtid, email, and PhD program name.
2. HW1 will be released by tomorrow. It will be due Sep 3.

2 What is this course about?

An *algorithm* is a sequence of instructions that accomplishes a task. We usually want more than correctness: we also want to minimize the resources used, such as time, space, or communication.

1. There is no single idea that solves every algorithmic problem. Instead, we will develop general approaches that apply to broad classes of problems;
2. learn useful abstractions and mathematical/proof techniques;
3. develop a theoretical way of thinking (a.k.a. theoretical lens) about new questions: useful even beyond computer science in fields, e.g., in fields such as economics, mathematics, and physics.

During the course, you are encouraged to use AI to learn and self-evaluate. The goal is to be curious, always learning, and develop skills to learn new things quickly.

Course roadmap

Part I Divide and conquer; dynamic programming; graph algorithms.

Part II Linear programming; convex optimization and gradient descent; approximation algorithms.

Part III Randomized algorithms; complexity theory.

Most communication, homework, and class notes will be posted on Piazza. Please consult the current syllabus on Canvas for grading policies, office hours, and other administrative details.

3 Asymptotic notation

For an input of size n, let $T(n)$ (or more generally $f(n)$) denote the amount of a resource used by an algorithm. We typically count *elementary operations* (like addition, subtraction, multiplication, and bit operations) and focus on how this quantity grows as the integer n becomes large.

3.1 Big-$\mathcal{O}$ notation: asymptotic upper bounds

Definition (Big-$\mathcal{O}$). Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We write

$$f(n) = \mathcal{O}(g(n))$$

if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq c\, g(n) \qquad \text{for every } n \geq n_0.$$

Thus, g is an eventual upper bound on f, up to a constant factor. When the limit exists, a useful test is

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty \quad \Longrightarrow \quad f(n) = \mathcal{O}(g(n)).$$

Example. We have $n^2 + 2n = \mathcal{O}(n^2)$: for $n \geq 1$, $n^2 + 2n \leq 3n^2$. We also have $n^2 + 2n = \mathcal{O}(n^3)$. Big-$\mathcal{O}$ is an upper bound, not necessarily the tightest upper bound.

3.2 Lower bounds, strict bounds, and tight bounds

Definition (Big-Ω). We write $f(n) = \Omega(g(n))$ if $g(n) = \mathcal{O}(f(n))$. Equivalently, there are constants $c > 0$ and n_0 such that $f(n) \geq c\, g(n)$ for every $n \geq n_0$.

Definition (Little-o and Little-ω). We write $f(n) = o(g(n))$ if, for every constant $c > 0$, there is an n_0 such that $f(n) < c\, g(n)$ for every $n \geq n_0$. For positive g, this is equivalent to

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0.$$

Intuitively, f grows strictly more slowly than g.

Similarly, $f(n) = \omega(g(n))$ means $g(n) = o(f(n))$.

Definition (Big-Θ). We write $f(n) = \Theta(g(n))$ if both $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$.

Some useful comparisons are

$$1 = o(\log\log n) = o(\log n) = o(n^{0.1}) = o(n) = o(2^n) = o(2^{n^2}).$$

Also, $n! = o(n^n)$, since

$$\frac{n^n}{n!} = \prod_{i=1}^{n} \frac{n}{i} \longrightarrow \infty.$$

Exercise. Prove that

$$\log(n!) = \Theta(n \log n).$$

One route is to compare $n!$ with n^n for the upper bound and to use only the largest $n/2$ factors in the product for the lower bound.

4 Divide and Conquer

The divide-and-conquer paradigm has three steps:

1. **Divide:** break the problem into smaller subproblems.
2. **Conquer:** solve each subproblem recursively.
3. **Combine:** assemble the subproblem solutions into a solution to the original problem.

5 Case study: merge sort

Input. An array $A[0], A[1], \ldots, A[n-1]$ of n numbers. (Here we are ignoring bit complexity; so imagine they are all 64-bit numbers and can be compared/rearranged in $\mathcal{O}(1)$ time.)

Goal. Rearrange the numbers in nondecreasing order.

For simplicity, assume that n is a power of two. This assumption is not essential: one may split using floors and ceilings, or conceptually pad the array.

5.1 Algorithm

1. If $n \leq 1$, return A.
2. Divide A into its left and right halves.
3. Recursively sort each half.
4. Merge the two sorted halves.

The merge step repeatedly compares the first unmerged element of each half, moves the smaller element to the output, and finally appends the unused suffix of the nonempty half.

Exercise. Show that two sorted arrays, each of length n, can be merged into one sorted array of length $2n$ in $\mathcal{O}(n)$ time.

Correctness. Correctness follows by induction on n. We will discuss induction in detail in next lecture, but briefly: The base case $n \leq 1$ is immediate. For the inductive step, the recursive calls correctly sort the two halves by the inductive hypothesis. The merge procedure outputs their elements in nondecreasing order and outputs every input element exactly once. Therefore the result is a sorted permutation of the original array.

5.2 Running time

Let $T(n)$ denote the worst-case running time on an input of length n. The two recursive calls operate on inputs of size $n/2$, and the merge step takes cn time for some constant $c > 0$. Thus

$$T(n) = 2T(n/2) + cn, \qquad T(1) = \Theta(1).$$

We can solve this recurrence using a recursion tree. At level i, there are 2^i subproblems, each of size $n/2^i$:

Level	Number of subproblems	Merge work per subproblem	Total merge work
0	1	cn	cn
1	2	$cn/2$	cn
2	4	$cn/4$	cn
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$cn/2^i$	cn

The subproblem size reaches 1 after $\log_2 n$ levels. Every non-leaf level does cn work, and the n leaves contribute $\Theta(n)$ total work. Hence

$$T(n) = cn \log_2 n + \Theta(n) = \Theta(n \log n).$$

Takeaway. Divide and conquer can turn a problem into many simpler copies of itself. Its efficiency depends on both the size and number of the subproblems and on the cost of combining their solutions.